DevBridge

Project Blueprint & Scope Control Document

Core vision: DevBridge is a developer transformation layer that converts one development artifact into another useful developer artifact.

Current status: TypeScript monorepo and shared IR are established; JSON → TypeScript is being finalized and tested.

1. The Problem

• Developers repeatedly rewrite schemas, models, API requests, diagrams, and other artifacts by hand.

• Existing tools are fragmented: one tool converts one format, another handles a different workflow, and developers repeatedly copy/paste between them.

• Boilerplate generated from existing schemas consumes time and can introduce inconsistent naming or types.

• Developer artifacts can contain proprietary or sensitive information, so deterministic transformations should preferably run locally.

DevBridge's job is simple: **take an artifact a developer already has and produce the artifact they need next.**

2. Product Concept

The key architectural decision is to avoid building a separate converter for every source/target pair.

```
Source → Parser → Shared Intermediate Representation (IR) → Generator → Target

Example: JSON → Parser → IR → TypeScript Generator → TypeScript interfaces
```

The same IR can then feed Java, Go, Python, and C# generators without rewriting the JSON parser.

3. Target Users

• Backend and full-stack developers.

• Frontend developers who need typed models or API artifacts.

• Developers working across multiple languages or technology stacks.

• Teams that want repeatable transformations in browser, IDE, CLI, or CI workflows.

4. Architecture

Repository: TypeScript + pnpm monorepo.

Core: @devbridge/core — shared IR and TransformationContext.

Transformers: JSON now; SQL and OpenAPI planned.

Applications: web app and Chrome extension; VS Code extension later.

Backend: Spring Boot only when authentication, subscriptions, cloud history, public API, or AI features actually require a server.

5. Current IR Scope

• Primitive: string, number, boolean, null, unknown.

• Object: named object with properties.

• Array: typed collection.

• Union: multiple possible types.

• Optional properties.

• TransformationContext for registered object types.

• Contextual naming to avoid collisions between nested objects with identical property names.

6. Development Phases

Phase	Goal	Status
0 — Foundation	GitHub repo, pnpm monorepo, TypeScript, core package, tests.	DONE
1 — JSON → TypeScript	Parser + IR + generator + edge-case tests.	CURRENT / NEAR DONE
2 — JSON → Java	Second generator proving IR reuse.	NEXT
3 — JSON targets	Go, Python, C# generators.	PLANNED
4 — Web MVP	Editor, target selector, validation, output, copy/download, local processing.	PLANNED
5 — SQL family	SQL → database model / ERD / DBML / Mermaid.	PLANNED
6 — OpenAPI family	OpenAPI → Postman and related developer artifacts.	PLANNED
7 — Integrations	Chrome extension, VS Code extension, CLI/API direction.	PLANNED
8 — Productization	Usage validation, accounts, premium/team/API features.	LATER
9 — AI assistance	Repair, explanation, suggestions, and difficult transformations.	LATER

7. Exact MVP Scope

• A developer pastes or loads an artifact.

• DevBridge validates it and reports useful errors.

• The transformation runs locally when deterministic processing is possible.

• The developer selects a target format.

• The generated result can be copied or downloaded.

• The first MVP should contain a small number of excellent transformations, not dozens of mediocre ones.

• Inputs are not stored by default.

• Every supported transformation has automated tests.

8. JSON → TypeScript Definition of Done

• Nested objects become named interfaces.

• Arrays of objects become reusable item interfaces.

• Objects in arrays are merged.

• Missing properties become optional.

• Primitive arrays are generated correctly.

• Mixed arrays become union arrays.

• Nullable values become unions when multiple observed types require it.

• Empty arrays become unknown[].

• Nested objects inside array items work.

• Object naming collisions are avoided.

• Regression tests cover the important edge cases.

9. Immediate Build Sequence

1	Finish JSON → TypeScript	Complete the remaining optional nested-object test and lock the milestone.
2	Build JSON → Java	Reuse the exact JSON parser + IR; initially only Java generation is new.
3	Define Java rules	Classes/records, naming, List types, nested types, nullability, imports/packages.
4	Test Java against shared fixtures	Use representative JSON cases already used for TypeScript.
5	Add Go/Python/C#	Only after the multi-generator abstraction is proven.
6	Build Web MVP	Expose the proven engine through a simple developer workflow.

10. Transformation Roadmap

Source	Targets / Outputs	Why it matters
JSON	TypeScript, Java, Go, Python, C#	First proof of shared IR and multi-language generation.
SQL	ERD, DBML, Mermaid, database model	Turn database definitions into diagrams/documentation.
OpenAPI	Postman and related API artifacts	Reduce manual API-tool setup.
Figma	React and frontend artifacts	Later exploration of design-to-code transformation.

11. Explicitly Out of Scope for Now

• Full SaaS accounts, teams, billing, and dashboards.

• Large AI infrastructure before deterministic transformations are proven.

• Dozens of converter types simultaneously.

• A server backend for transformations that can safely run locally.

• Public API before the core transformation contracts stabilize.

• Mobile application.

• Complex collaboration/social features.

• Persistent storage of transformation inputs by default.

12. Quality & Security Rules

• **Correctness over quantity.** Reliable transformations are more important than a large catalog.

• **IR-first.** New targets should reuse the shared representation whenever practical.

• **Regression tests.** Every important parser/generator bug becomes a test.

• **Local-first.** Sensitive deterministic transformations should not require uploading source data.

• **Explicit AI.** AI-powered behavior should be clearly separated from deterministic conversion.

• **Developer UX.** Results should be immediately usable.

• **Scope discipline.** A feature must strengthen either the transformation engine or the developer workflow around it.

13. Productization Later

• Free browser/local transformations for common use cases.

• Premium developer integrations and advanced generators.

• API usage plans for automated transformations.

• Team/organization capabilities.

• AI-assisted transformations as a possible paid capability.

Monetization should follow evidence of developer usage rather than drive the architecture prematurely.

14. Success Criteria

• A developer can turn a real artifact into a useful target artifact within seconds.

• Multiple generators can consume the same IR.

• Adding a target is primarily a generator task, not another complete converter.

• The engine can eventually power web, extension, CLI, and API experiences.

• Real developers use the MVP and reveal which transformations deserve further investment.

15. The Rule We Follow From Here

```
Core → Parser → IR → Generator → Tests → Second Generator → More generators → Web MVP →
More transformation families → Integrations → Monetization → AI
```

Whenever we consider a new feature, ask: **Does this strengthen the transformation engine or the developer workflow around it?** If not, it is probably outside the current scope.

This document is the project's scope anchor. We can update it when a deliberate product decision changes the roadmap.